# RIPHAH INTERNATIONAL UNIVERSITY

## COMPUTER ORGANIZATION AND ASSEMBLY LANGUAGE

## LAB 10 TASK 3

Name: Aroosa Feroz

SAP ID: 68997

Submitted to: Ma'am Nehan Sahar

Subject: Computer Organization and Assembly Language

**DEPARTMENT OF COMPUTER SCIENCE**

**RIPHAH INTERNATIONAL UNIVERSITY, GULDBERG GREENS CAMPUS ISLAMABAD**

# Task 3: Explanation required

```
.model small
.stack 100h
.data
str db 'HELLO WORLD$'

.code
main proc

   mov ax, @data
   mov ds, ax

   mov si, offset str

; 🔻 Step 1: Move SI to end of string
find_end:
   mov al, [si]
   cmp al, '$'
   je start_reverse
   inc si
   jmp find_end

; Move back from '$'
start_reverse:
   dec si

; 🔻 Step 2: Process words in reverse
reverse_loop:
   cmp si, offset str
   jb done

   mov cx, 0            ; word length counter

; 🔻 Count characters of word
count_word:
   cmp si, offset str
   jb push_word
   mov al, [si]
   cmp al, ' '
   je push_word
   inc cx
   dec si
   jmp count_word

; 🔻 Push word characters into stack
push_word:
   mov bx, cx
   mov di, si
   inc di

push_chars:
   cmp bx, 0
   je print_word
   mov al, [di]
   mov ah, 0
   push ax
   inc di
   dec bx
   jmp push_chars

; 🔻 Pop and print characters
print_word:
   mov bx, cx

print_chars:
   cmp bx, 0
   je print_space
   pop dx
   mov ah, 2
   int 21h
   dec bx
   jmp print_chars

; 🔻 Print space between words
print_space:
   mov dl, ' '
   mov ah, 2
   int 21h

   dec si
   jmp reverse_loop

done:
   mov ah, 4ch
   int 21h
main endp
end main
```

Ans:

This given program reverses the order of words in a string using a stack!

Example:
HELLO WORLD - WORLD HELLO
The major concept used is Stack (LIFO - Last In First Out)
Last pushed character - first popped

This is why stack helps in reversing

**1. Data Initialization**

- Code:

```
mov ax, @data
mov ds, ax
```

- Explanation:

Loads data segment address into DS register
Required to access variables like str

**2. Load String Address**

- Code:

```
mov si, offset str
```

- Explanation:

SI points to start of string
Example:
SI -> HELLO WORLD $

**3. Find End of String ($)**

- Code:

```
find_end:
mov al, [si]
cmp al, '$'
je start_reverse
inc si
jmp find_end
```

- Explanation:

Loop runs until $ is found
$ is string terminator in DOS
After loop:
SI -> $

**4. Move One Step Back**

- Code:

```
start_reverse:
```

dec si

- Explanation:

Move from $ to last character
Now:
SI -> D (last character)

**MAIN LOGIC**

**5. Reverse Loop**

- Code:

```
reverse_loop:
   cmp si, offset str
   jb done
```

- Explanation:

Continue until beginning of string
If SI < start then program ends

**6. Reset Word Counter**

- Code:

```
mov cx, 0
```

- Explanation:

CX will count characters of one word

**7. Count Characters of a Word**

- Code:

```
count_word:
   cmp si, offset str
   jb push_word
   mov al, [si]
   cmp al, ' '
   je push_word
   inc cx
   dec si
   jmp count_word
```

- Explanation:

Moves backward
Stops when:
Space ' ' found OR
Start of string reached
Example:
HELLO WORLD
                ↑ start here

It counts: D, L, R, O, W i,e CX = 5

**8. Prepare to Push Word**

- Code:

```
push_word:
mov bx, cx
mov di, si
inc di
```

- Explanation:

BX = CX i.e number of characters
DI points to start of word
Example:
DI is W in World

**9. Push Characters into Stack**

- Code:

```
push_chars:
   cmp bx, 0
   je print_word
   mov al, [di]
   mov ah, 0
   push ax
   inc di
   dec bx
   jmp push_chars
```

- Explanation:

Push each character of word into stack
One by one
Stack after pushing "WORLD" will be as:
TOP D
L
R
O
W

**10. Pop and Print Characters**

- Code:

```
print_word:
   mov bx, cx
print_chars:
   cmp bx, 0
   je print_space
   pop dx
   mov ah, 2
   int 21h
   dec bx
   jmp print_chars
```

- Explanation:

Pops characters i.e prints using DOS interrupt
Output:
WORLD
Because:
Stack reverses twice and original word restored
BUT word position is reversed

**11. Print Space**

- Code:

```
print_space:
   mov dl, ' '
   mov ah, 2
   int 21h
```

- Explanation:

Adds space between words

**12. Move to Previous Word**

- Code:

```
dec si
jmp reverse_loop
```

- Explanation:

Move backward to next word

**13. Program End**

- Code:

```
   mov ah, 4ch
   int 21h
```

- Explanation:

Terminates program

**FULL WORKING EXAMPLE**

Input:
HELLO WORLD

Start from end i.e WORLD
Push WORLD then print WORLD
Move to HELLO
Push HELLO then print then HELLO

Push first then printed that causes reversal

Output: WORLD HELLO